Table of Contents

CHAPTER 1: INTRODUCTION……1

1.1 Project Definition…….....1

1.2 Objective……1

1.3 Tools & Technologies…….……..2

CHAPTER 2: LITERATURE REVIEW……..3

2.1 Introduction…….3

2.2 Existing Systems…….3

2.3 Gaps in Existing Systems……....3

CHAPTER 3: SYSTEM ANALYSIS……...5

3.1 Functional Requirements…….5

3.2 Non-Functional Requirements.....6

CHAPTER 4 : TECHNOLOGY STACK……..8

4.1 Frontend(Client-Slide)…….....8

4.2 Backend(Server-Slide)…….....8

4.3 Database……...9

CHAPTER 5: SYSTEM DESIGN……...10

CHAPTER 6: TESTING…….. .12

CHAPTER 7: RESULTS..... 13

CHAPTER 8: CHALLENGES FACED……..20

CHAPTER 9 : CONCLUSION AND FUTURE SCOPE……....21

9.1 Conclusion…….…21

9.2 Future Scope……...21

REFERENCES……..……22

LIST OF FIGURES

Figure 1: Use Case Diagram…….1

Figure 2: Flow Chart Diagram…….....10

Figure 3: ER Diagram……...11

Figure 4: Register Page…….14

Figure 5: Login Page…….14

Figure 6: DashBoard…….…….15

Figure 9: Health Analyzer page….……..15

Figure 10: Mental Health screen…..……16

Figure 11: Diabeties prediction…….…….…..16

Figure 12: Heart attack Prediction page…….…..17

Figure 13: Water Tracker……..…….…..17

Figure 14: Nutrition/Food logger…….……...17

Figure 15: Streak/badges…….18

Figure 16: Profile page……...…….…18

Figure 17: Ai chat……....19

Figure 18: Ai meal suggestion…….19

Figure 19: Calorie Calculator.…….19

ABSTRACT

Health awareness and early risk detection are critical in today's world, yet most people lack accessible tools to assess their health condition without visiting a doctor. This project, Smart Health Analyzer, is a full-stack web application built using React 19, FastAPI (Python), and Supabase (PostgreSQL) that enables users to predict health risks, track daily wellness habits, and receive AI-powered insights — all from a single platform.

The system provides ML-based predictions for three major health conditions: mental health disorders, diabetes, and heart disease, trained using scikit-learn models. Beyond prediction, users can track daily water intake, log meals and nutrition, monitor calorie and macro goals, and maintain streaks with a gamification system including badges and rewards.

The goal of Smart Health Analyzer is to create a personalized, data-driven wellness companion that makes health monitoring simple, intelligent, and engaging for everyday users. By combining machine learning predictions with habit tracking and AI coaching, the platform empowers users to take proactive control of their health.

CHAPTER 1: INTRODUCTION

1.1 Project Definition

Smart Health Analyzer is a full-stack health monitoring and risk prediction platform designed to make preventive healthcare accessible to everyone. It is a web application that combines machine learning-based health condition screening with daily wellness tracking features such as water intake, nutrition logging, meal planning, and streak-based gamification.

Unlike generic fitness apps, **Smart Health Analyzer** uses trained ML models to screen users for mental health conditions, diabetes risk, and heart disease risk based on clinical parameters they input. The system stores all predictions, health logs, and user history in a cloud database with row-level security, ensuring data is private and persistent across sessions. A gamification layer — including daily streaks, milestone badges, and progress tracking — encourages users to stay consistent with their health habits over time.

1.2 Objective

The primary objective of the **Smart Health Analyzer** project is to provide a comprehensive, intelligent, and accessible platform for personal health monitoring and risk prediction.

The specific objectives are:

- To develop ML-based screening tools for mental health, diabetes, and heart disease using scikit-learn.
- To build a daily water intake tracker with hydration goal setting and logging.
- To provide a nutrition and meal diary with calorie and macro tracking capabilities.
- To implement a gamification system with daily streaks, streak logs, and achievement badges.
- To enable secure, persistent user authentication and profile management using Supabase.
- To store all health records, predictions, and logs with row-level security policies in PostgreSQL.
- To design a clean, mobile-friendly interface targeting everyday users rather than medical professionals.
- To deploy a scalable full-stack system with the frontend on Vercel and backend on Render.

1.3 Tools & Technologies

1.3 Tools & Technologies

The **Smart Health Analyzer** application is built using a modern full-stack architecture combining a React frontend, a Python FastAPI backend, and Supabase for database and authentication.

Frontend (Client-Side)

- **React 19 (Vite):** For building a fast, component-based dynamic user interface.
- **Tailwind CSS:** For creating a clean, responsive design with a custom Anti-Gravity design system using deep space navy, electric teal, and violet color palette.
- **React Router v7:** For client-side routing across all pages of the application.
- **Axios:** For making HTTP requests from the frontend to the FastAPI backend.

Backend (Server-Side)

- **FastAPI (Python):** A high-performance Python web framework used to build REST API endpoints for health prediction and data processing..
- **scikit-learn:** The machine learning library used to train and serve prediction models for mental health, diabetes, and heart disease.
- **NumPy & Pandas:** Used for data manipulation and feature processing in the ML pipeline.

Database & Authentication

- **Supabase (PostgreSQL):** A cloud database with built-in authentication, providing secure user management and data persistence.
- **Row-Level Security (RLS):** Supabase RLS policies ensure each user can only access their own health data.

Deployment

- **Vercel:** Frontend deployment with automatic CI/CD from GitHub.
- **Render:** Backend (FastAPI) deployment on a free-tier server.

CHAPTER 2: LITERATURE REVIEW

2.1 Introduction

The literature review for Smart Health Analyzer involves studying existing health monitoring applications, wellness platforms, and ML-based diagnostic tools. By examining what is currently available, we can identify the technical and usability gaps that motivated the creation of this platform.

For this project, various types of health apps — from symptom checkers to fitness trackers — were analyzed. This study helped in understanding how health data is typically collected, processed, and presented to users, and provided a clear direction for building a more integrated and intelligent health companion.

2.2 Existing Systems

Several categories of applications were studied during the research phase:

- **Generic Fitness Apps** — Applications like MyFitnessPal and Google Fit track physical activity and nutrition but offer no clinical risk screening. They rely entirely on user-input data without providing any predictive health insights based on medical parameters.
- **Symptom Checkers** — Platforms like WebMD Symptom Checker or Ada Health allow users to input symptoms and receive possible diagnoses. However, they are rule-based systems with no machine learning models, and they do not track habits or provide ongoing health monitoring.
- **Standalone ML Health Tools** — Research prototypes and Kaggle notebooks demonstrate the use of machine learning for predicting diseases like diabetes and heart disease. These, however, exist only as isolated scripts with no real user interface, authentication, or data persistence.
- **Mental Health Apps** — Applications like Wysa or Woebot focus specifically on mental wellness through chatbot-based conversations but lack integration with physical health metrics or medical risk prediction models.

While these systems are useful individually, no single platform combines clinical ML-based risk prediction with daily habit tracking, gamification, and persistent user history in one accessible web application.

2.3 Gaps in Existing Systems

After analyzing existing systems, the following key limitations were identified:

- **No Integration:** Existing tools are siloed — fitness apps don't predict disease risk, and symptom checkers don't track habits. Users must use multiple apps to manage their health holistically.
- **No Persistence for Predictions:** Most health screening tools do not save prediction history, so users cannot track how their risk changes over time.
- **Lack of Gamification:** There is little motivation built into existing health apps to encourage daily engagement, leading to users abandoning the habit of tracking.
- **No ML for Everyday Users:** ML-based health models are largely confined to academic settings and are not made accessible through user-friendly interfaces for non-technical users.
- **Weak Security:** Many health tracking apps do not implement proper row-level data security, exposing user health records to potential misuse.

To overcome these gaps, **Smart Health Analyzer** was developed with the following improvements:

- **Unified Platform:** A single application combining ML predictions, water tracking, nutrition logging, and streak gamification.
- **Persistent Prediction History:** All ML predictions are stored in Supabase with timestamps, allowing users to review and compare results over time.
- **Gamification Layer:** Daily streaks, milestone badges, and streak logs motivate consistent engagement.
- **Accessible ML Screening:** Three trained scikit-learn models are served through a FastAPI REST API with a clean React frontend anyone can use.
- **Row-Level Security:** Supabase RLS policies ensure every user's health data is completely private and isolated.

CHAPTER 3: SYSTEM ANALYSIS

3.1 Functional Requirements

Functional requirements define the specific features and services that **Smart Health Analyzer** must provide to the user.

User Authentication & Profiles

- Users can register and log in using email and password via Supabase Auth.
- Users can set up and edit their health profile including age, gender, height, and weight.
- Profile data is stored in the profiles table with RLS policies tied to user ID.

ML-Based Health Risk Prediction

- Users can input clinical parameters (blood pressure, cholesterol, glucose, smoking status, etc.) and receive a risk prediction.
- Three separate prediction models are available: Mental Health Screening, Diabetes Risk, and Heart Disease Risk.
- All predictions are logged and stored in the predictions and health record tables in Supabase.

Water Intake Tracking

- Users can log daily water consumption in increments and set a daily hydration goal.
- Water logs are stored in the water_logs table and displayed with progress visualization.

Nutrition & Meal Logging

- Users can log meals with details including food item, calories, protein, carbohydrates, and fat.
- The system maintains a meal diary and calculates daily totals for macro tracking.
- Nutrition data is stored in the nutrition_logs table.

Streaks & Gamification

- The system tracks daily engagement streaks based on user activity.
- Milestone badges are awarded when users reach streak targets (e.g., 7-day, 30-day streaks).
- Streak data is stored across streaks, streak_logs, and badges tables.

Prediction & Health History

- Users can view their full history of health predictions and wellness logs.
- All history is filterable by date and category.

3.2 Non-Functional Requirements

Non-functional requirements define the quality, performance, and reliability standards of **Smart Health Analyzer**.

Performance

- The FastAPI backend must respond to prediction requests within acceptable latency. ML models are loaded at startup to avoid per-request loading delays.
- The Render free-tier cold start delay is managed by displaying skeleton loading UI on the frontend.

Usability

- The interface targets everyday users with no medical background. All health metrics, risk levels, and tips are presented in plain, friendly language.
- The Anti-Gravity design system (dark navy background, teal and violet accents, glass-morphism cards) provides a modern, accessible visual experience.

Responsiveness

- The application is fully responsive with a BottomNav component for mobile navigation and appropriate layouts for desktop views.

Security

- Supabase Row-Level Security (RLS) policies enforce that users can only read and write their own health data.
- The profiles table uses id as its primary key for RLS comparisons; all other tables use user_id.
- No sensitive health data is stored on the client side.

Reliability & Data Persistence

- All user health logs, predictions, and streaks are persisted in Supabase PostgreSQL, ensuring data survives across sessions and devices.
- Auth loading state resolves before any navigation check runs, preventing premature redirects to the login page.

Scalability

- The decoupled architecture (React on Vercel, FastAPI on Render, Supabase as BaaS) allows each layer to scale independently.

Maintainability

- New features are added as new files/routes without modifying existing working code, following the project's core development principle.

CHAPTER 4: TECHNOLOGY STACK

4.1 Frontend (Client-Side)

The frontend is responsible for the user interface, routing, and communication with the backend API. It is designed to be fast, responsive, and visually distinctive.

- **React 19 (Vite):** Used to build a component-based, fast-rendering user interface. Vite provides near-instant hot module replacement during development.
- **Tailwind CSS:** Used for utility-first styling with a custom design system — deep space navy (#0B0E1A), electric teal (#00E5C3), and violet (#7B61FF) — creating a professional health-app aesthetic.
- **React Router v7:** Handles client-side navigation between pages including Login, Register, Dashboard, Analyze, Profile, Water Tracker, Food Logger, Streaks, and AI features.
- **Axios:** Used for all HTTP communication with the FastAPI backend, following a consistent api.js pattern for endpoint management.

It provides users with an intuitive, mobile-friendly experience for interacting with health tools and tracking their wellness data.

4.2 Backend (Server-Side)

The backend handles ML inference, business logic, and data processing via a clean REST API.

- FastAPI (Python): A modern, high-performance web framework for building the prediction and health data endpoints. Automatic OpenAPI documentation is generated at /docs.
- Uvicorn: The ASGI server that runs the FastAPI application, enabling asynchronous request handling.
- scikit-learn: The machine learning library used to train classification models for mental health screening, diabetes risk prediction, and heart disease risk prediction using clinical datasets.
- NumPy & Pandas: Used for feature engineering and data preprocessing in the ML pipeline before model inference.
- Pydantic: Handles request validation and ensures all incoming health data conforms to the expected schema before processing.

It ensures that ML predictions are served reliably, input data is validated, and the API is well-structured for frontend consumption.

4.3 Database

The database and auth layer provides persistent, secure storage for all user data, health records, and gamification data.

- **Supabase (PostgreSQL):** A Backend-as-a-Service platform that provides a managed PostgreSQL database with a real-time API. The schema includes 10 tables: profiles, predictions, mental_health_records, health_records, water_logs, nutrition_logs, streaks, streak_logs, badges, and meal_plans.
- **Supabase Auth:** Manages user registration, login, and session management using email/password. The session provides the user_id used in all data operations.
- **Row-Level Security (RLS):** PostgreSQL RLS policies on every table ensure strict data isolation — each user can only access rows belonging to their own account.

Together, they provide a secure, scalable, and persistent data layer that requires no separate backend database server to manage.

CHAPTER 5 : SYSTEM DESIGN

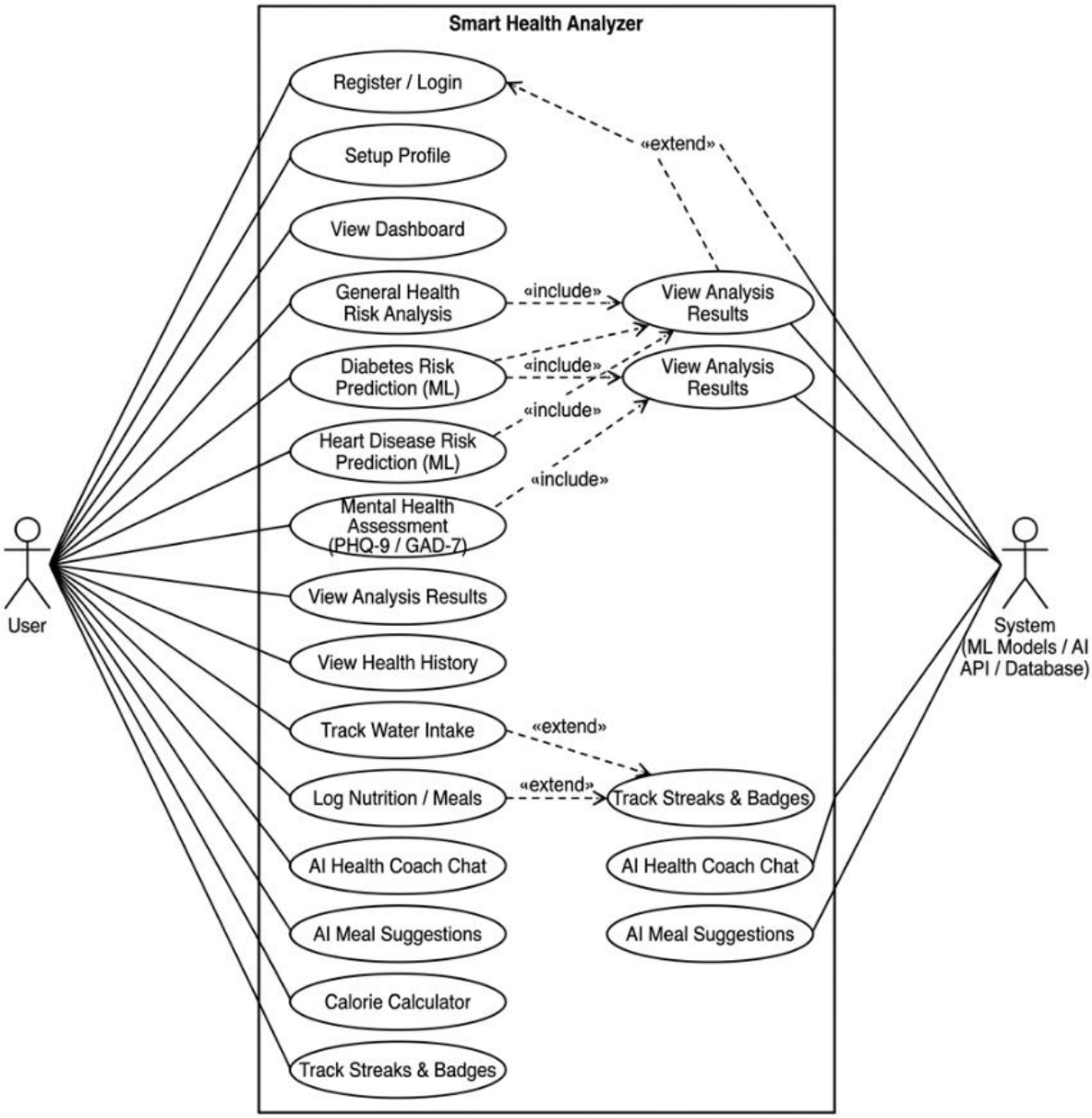


Figure 1 : Use Case Diagram

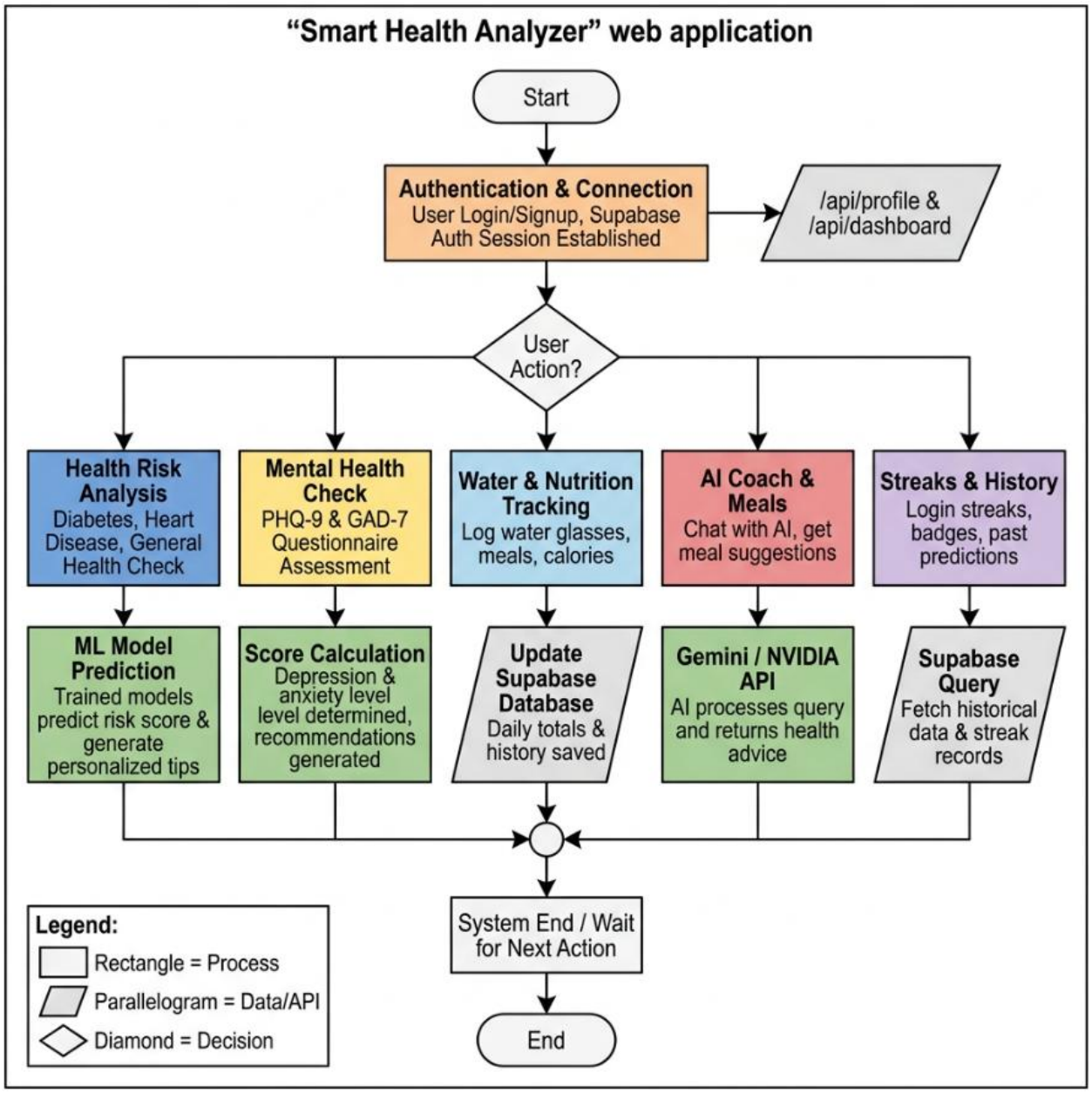


Figure 2 : Flow Chart

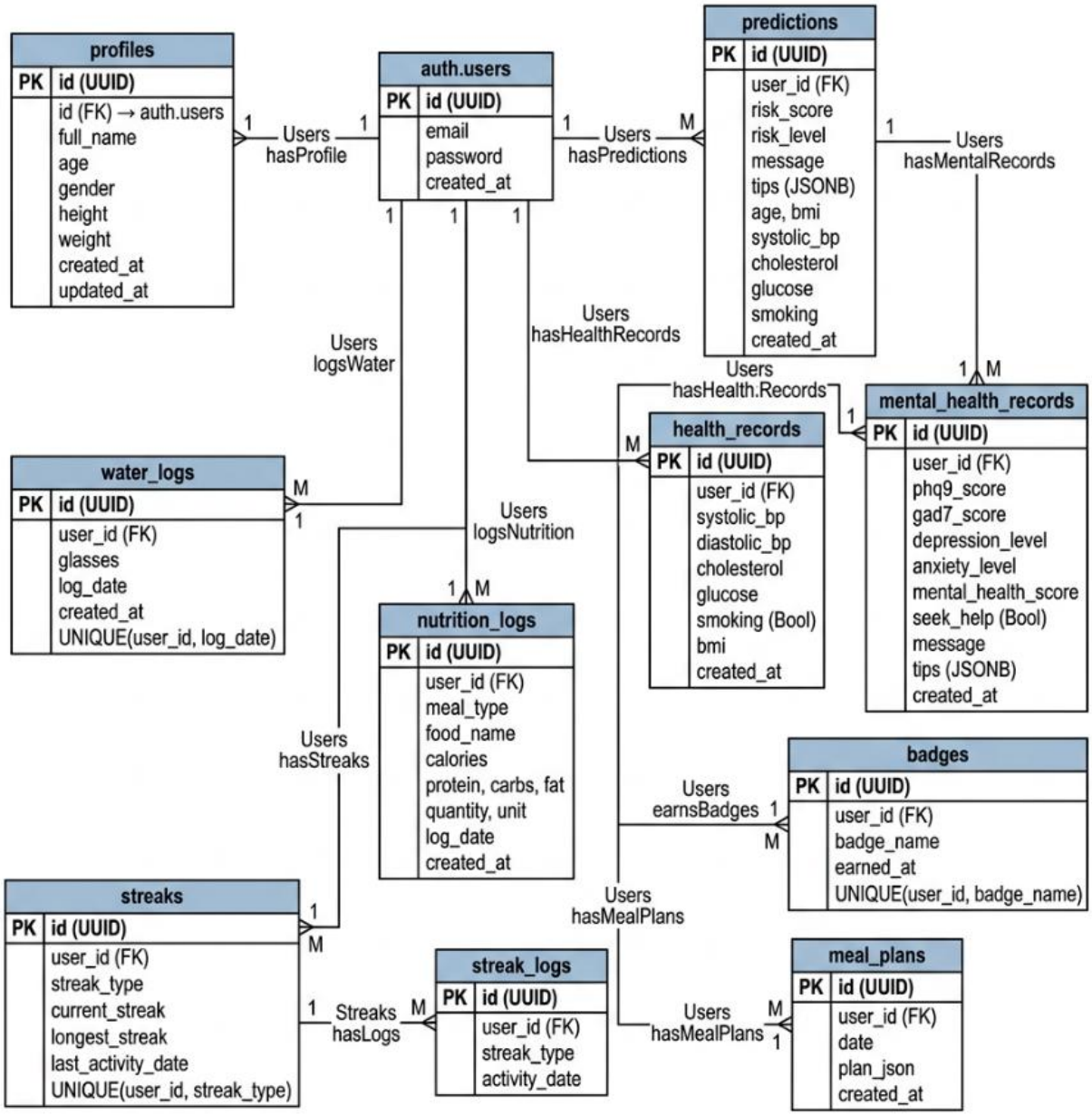


Figure 3 : ER Diagram

CHAPTER 6: TESTING

Testing was a critical phase in the development of **Smart Health Analyzer** to ensure prediction accuracy, data security, and a reliable user experience.

1. Unit Testing Unit testing involved testing individual components and functions in isolation. The FastAPI prediction endpoint (POST /predict) was tested with various combinations of clinical input parameters to verify that the rule-based scoring engine and ML model returned expected risk levels (Low, Medium, High) and appropriate health tips. Individual React components such as the water tracker logger and streak counter were tested independently to ensure correct state updates.

2. Integration Testing Integration testing verified that different layers of the system work together correctly. We tested the complete flow from the React frontend submitting health data via Axios, through the FastAPI backend processing the request, and the result being stored in Supabase and displayed back to the user. The Supabase RLS policies were also tested to confirm that users cannot read or write each other's data.

3. Functional Testing Functional testing was performed to verify every feature of **Smart Health Analyzer** works as intended. Key test cases included:

- **Health Prediction:** Confirming that ML models return correct risk levels for known input combinations.
- **Water Tracking:** Ensuring water log entries are saved, totals update correctly, and daily goals reset.
- **Nutrition Logging:** Verifying meal entries are persisted and macro totals calculate accurately.
- **Streak System:** Testing that streaks increment on daily activity, milestone badges are awarded correctly, and streak logs are recorded.
- **Auth Flow:** Confirming that registration, login, session persistence, and protected route guarding all work correctly without premature redirects.

4. Usability Testing Usability testing focused on the user experience for non-technical everyday users. We verified that health risk results are displayed in plain language with clear risk indicators (color-coded Low/Medium/High), that the BottomNav provides intuitive mobile navigation, and that the dark-themed Anti-Gravity design is readable and visually consistent across all pages.

5. Security Testing Security testing was conducted to protect sensitive health data. We verified that Supabase RLS policies prevent any cross-user data access, that unauthenticated users are redirected to the login page before auth state resolves, and that no health data is exposed in client-side storage or network requests without proper session authentication.

CHAPTER 7: RESULTS

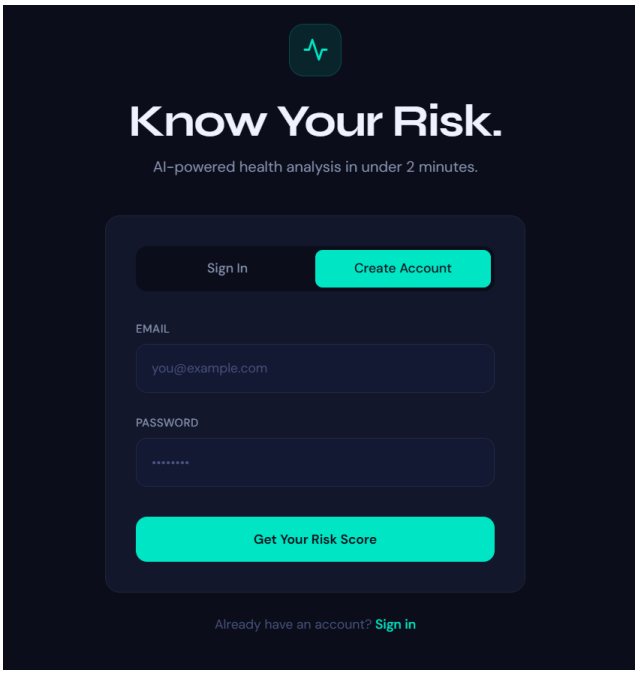


Figure 4 : Register page

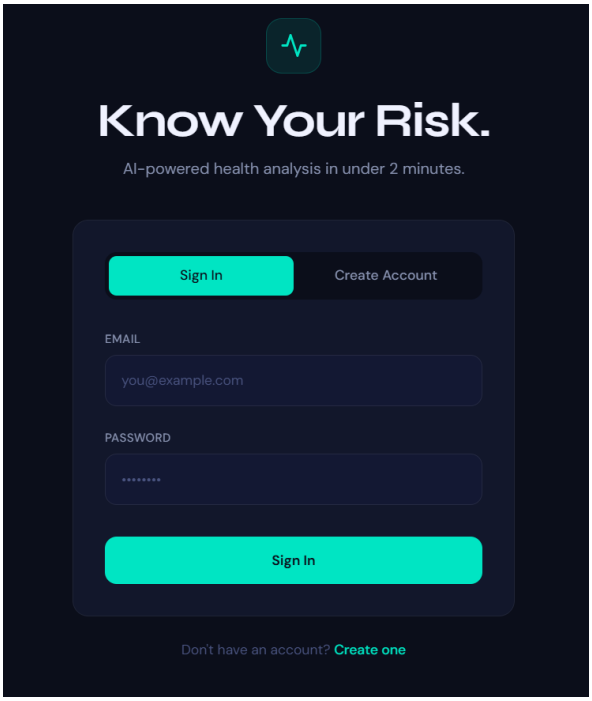


Figure 5 : Login Page

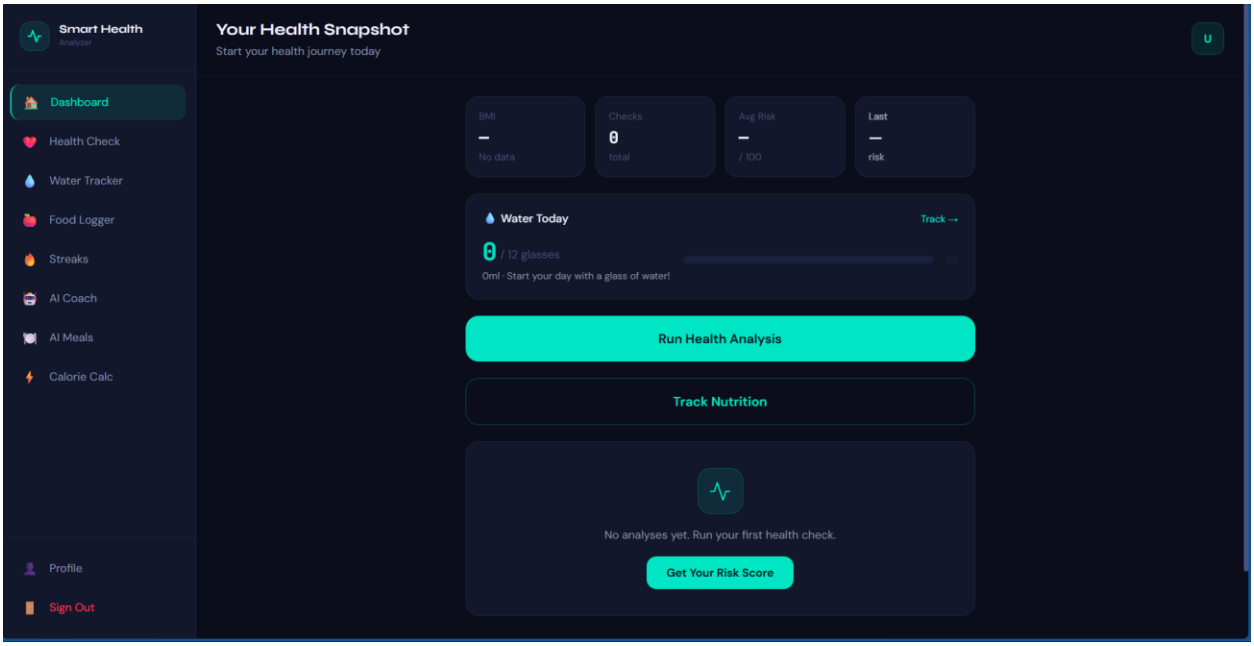


Figure 6 : Dashboard

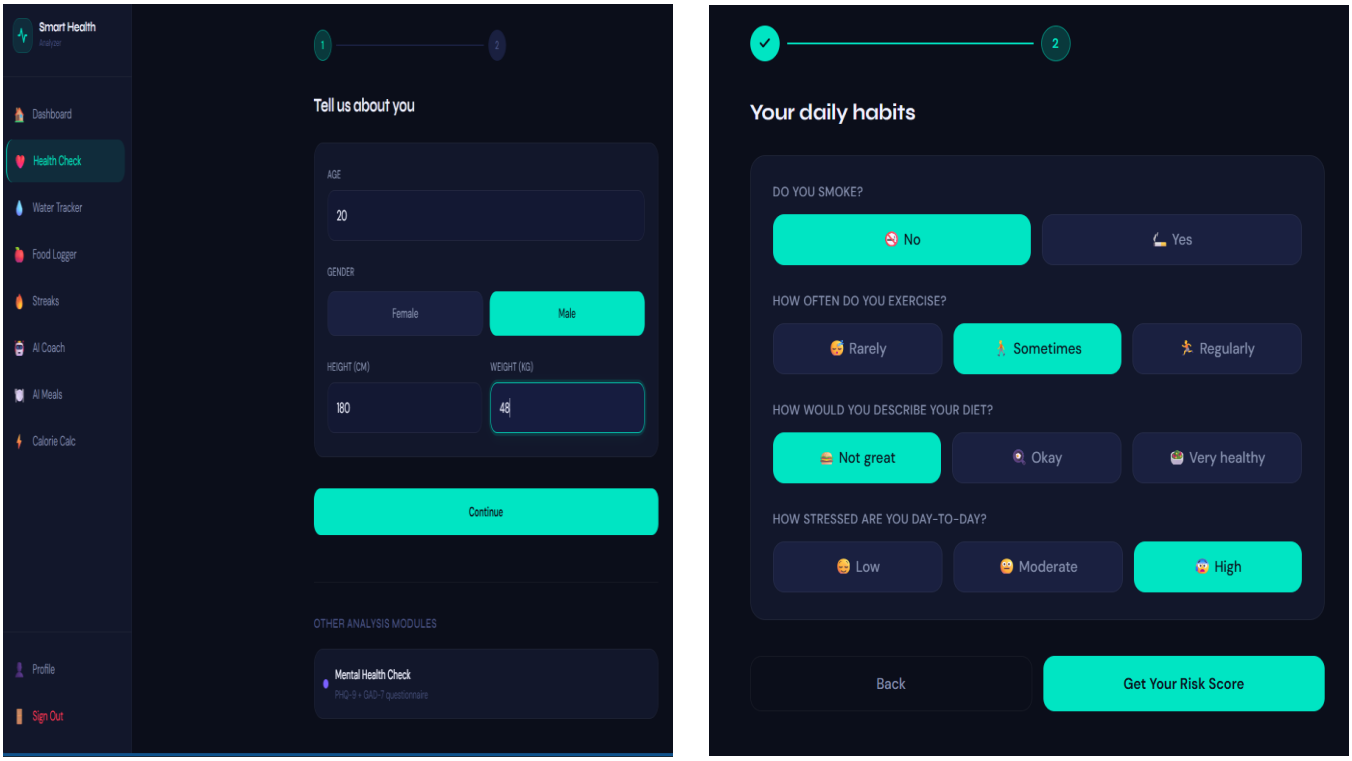
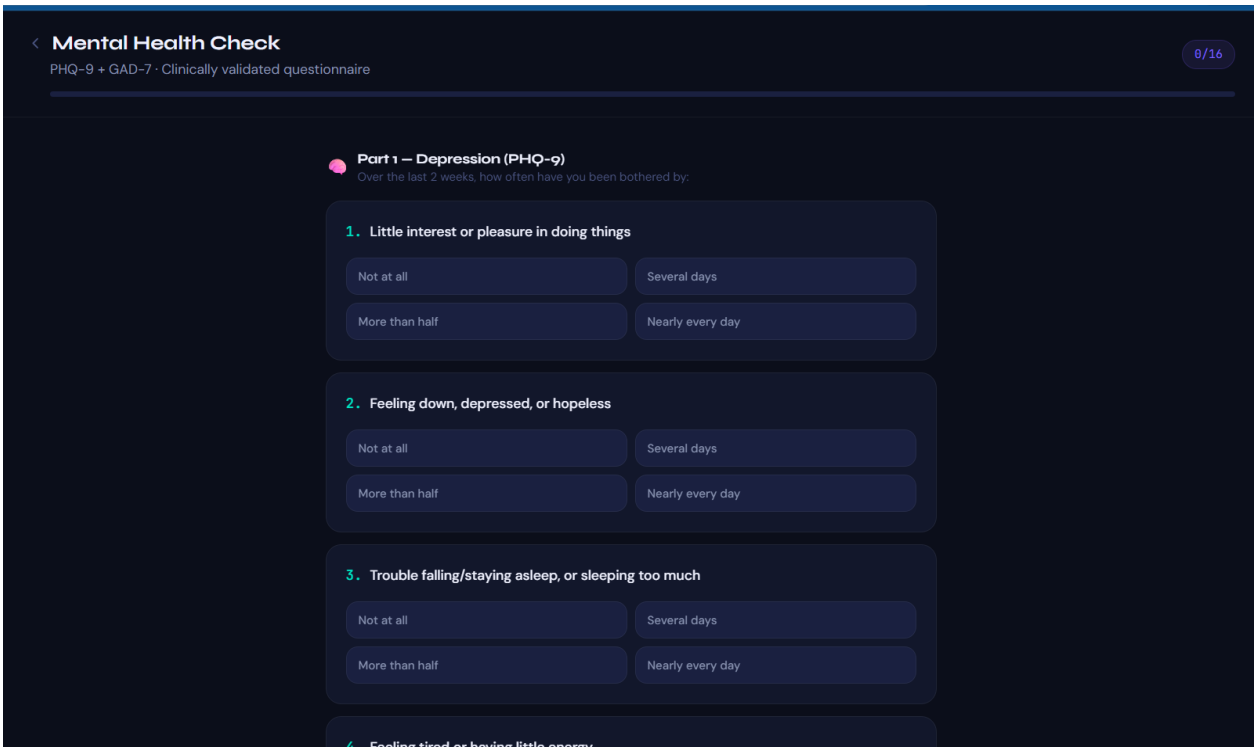


Figure 7 : Health ANalyzer



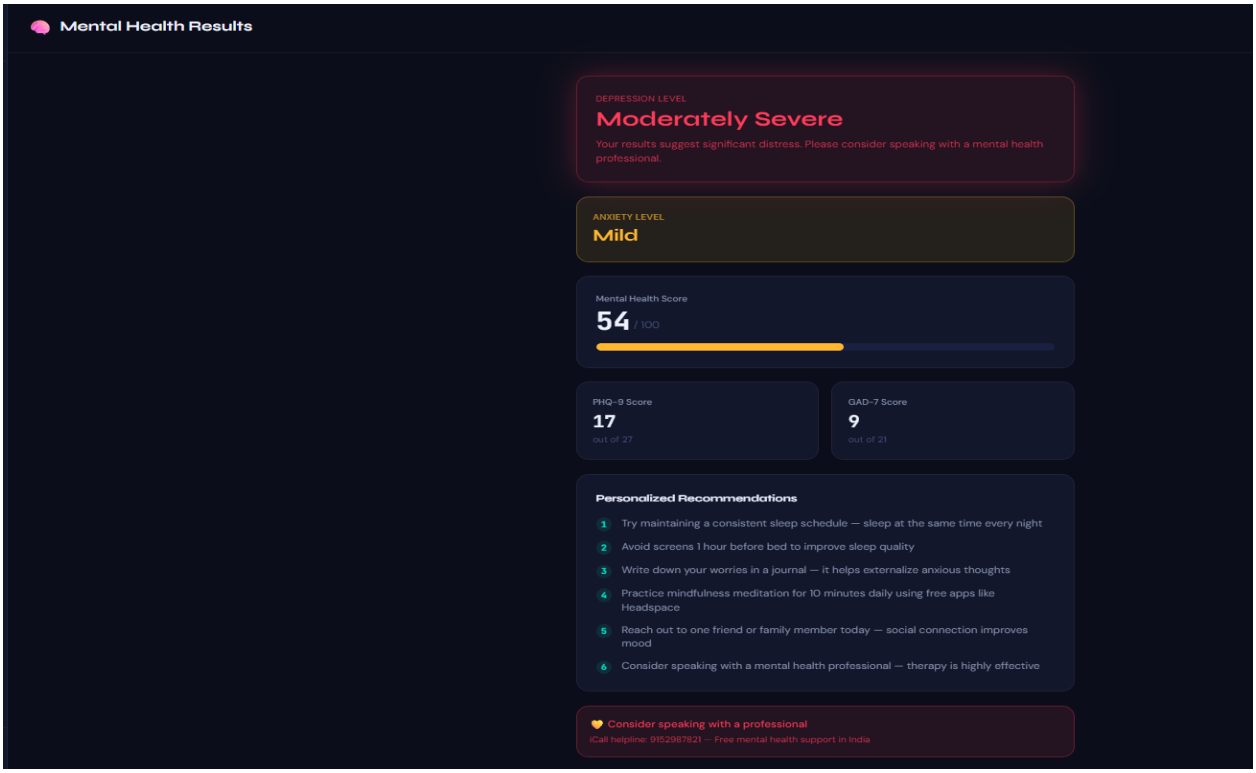
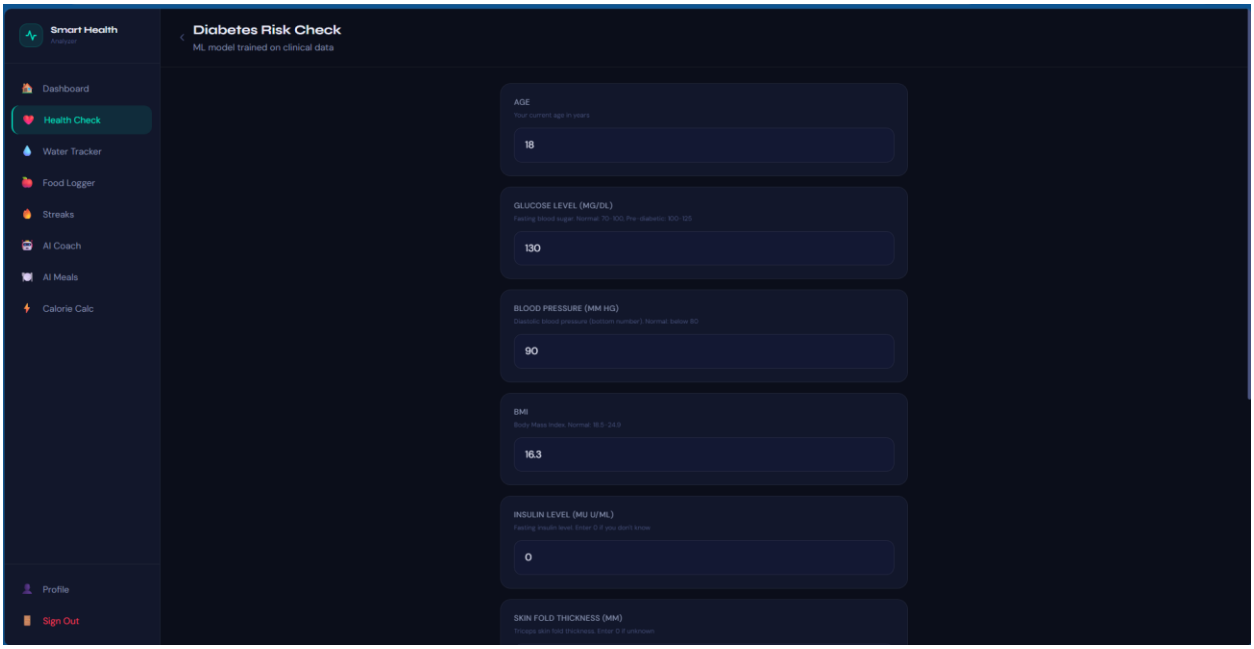


Figure 8 : Mental Health screen



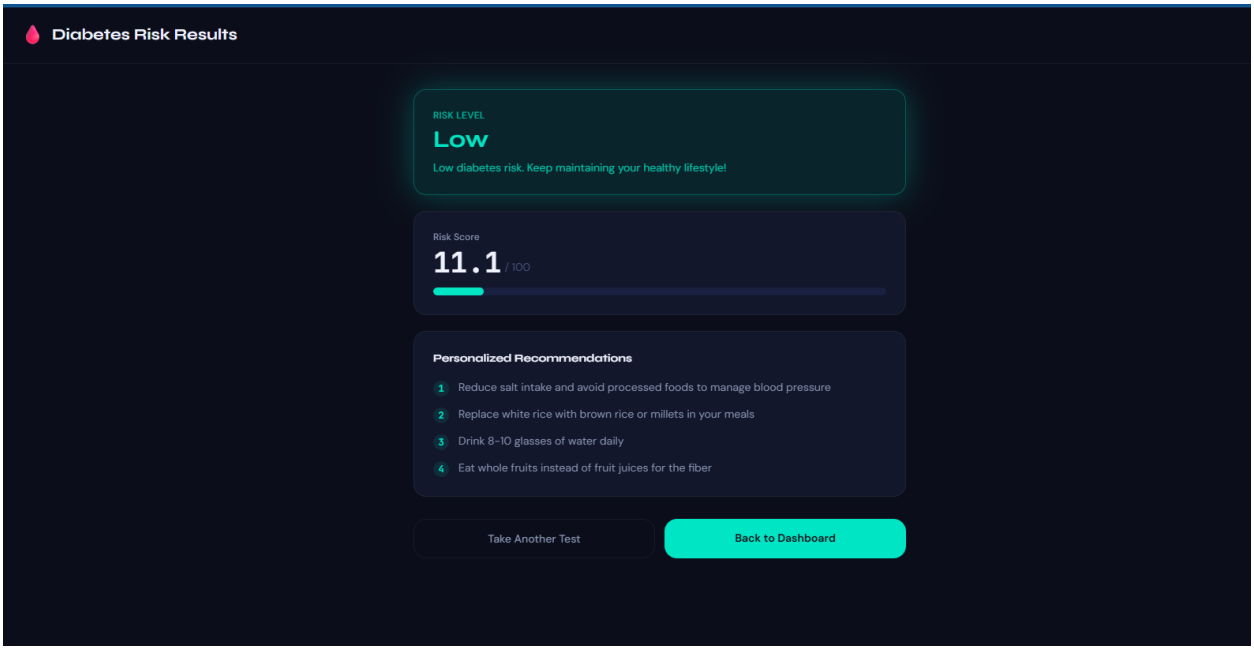
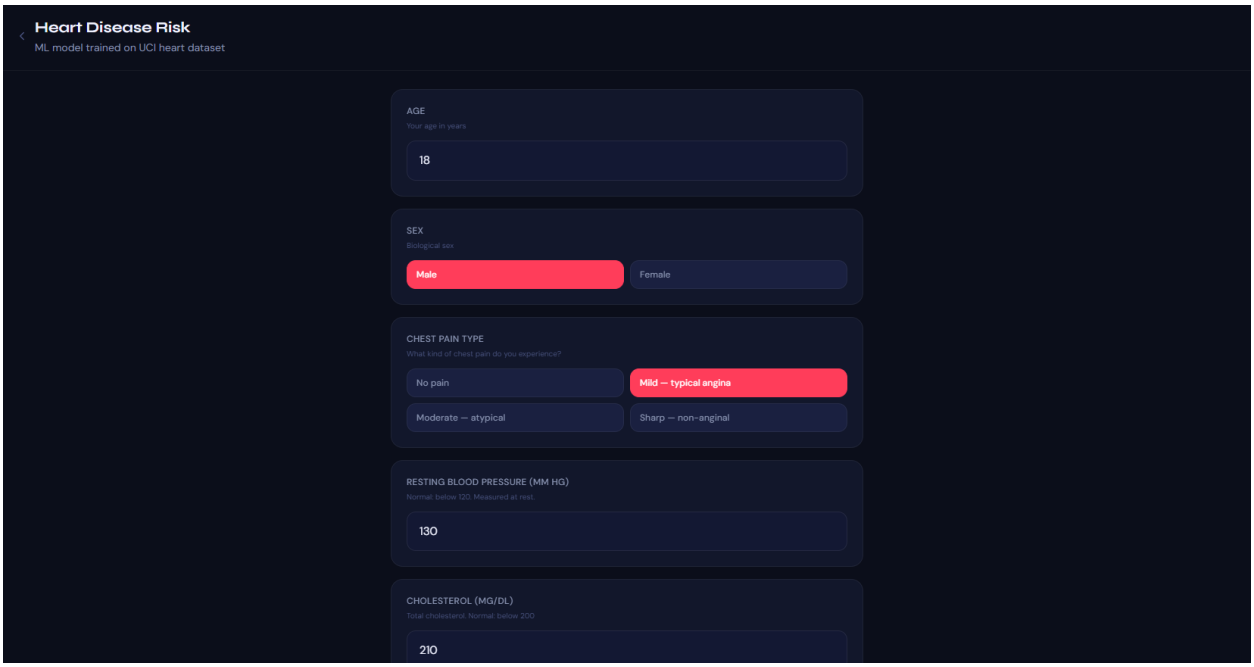


Figure 9 : diabiteas prediction page



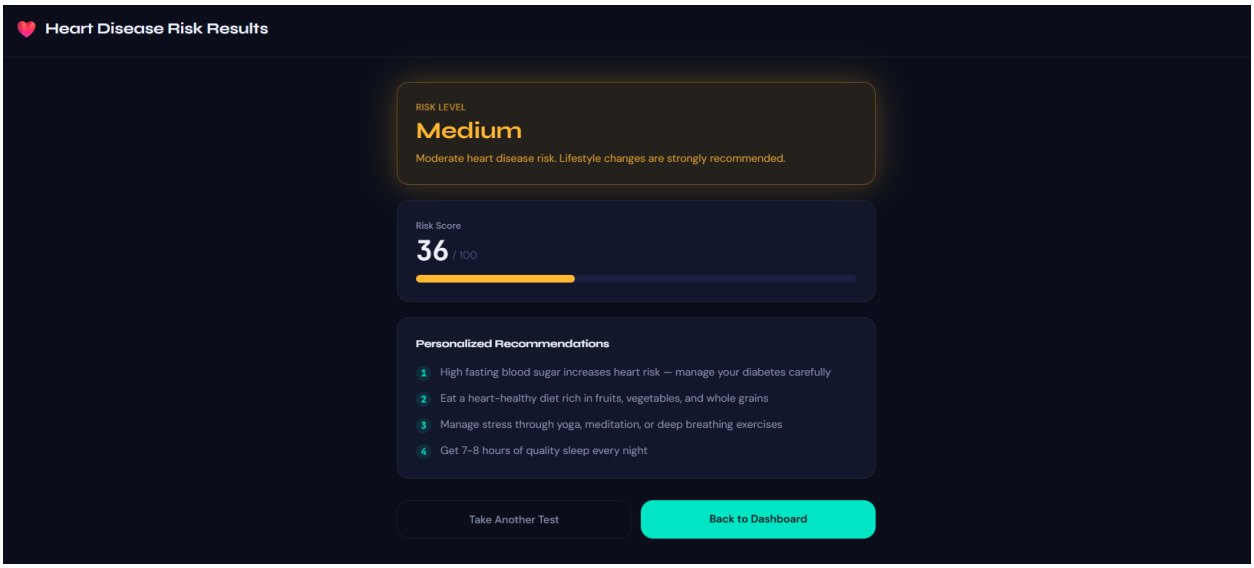


Figure 10 : Heart attack prediction page

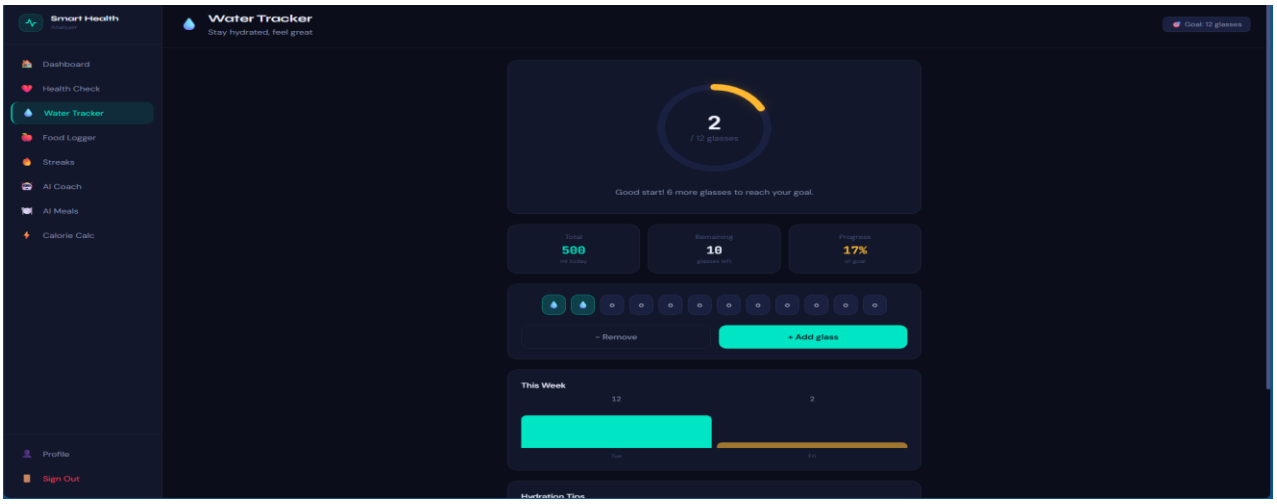
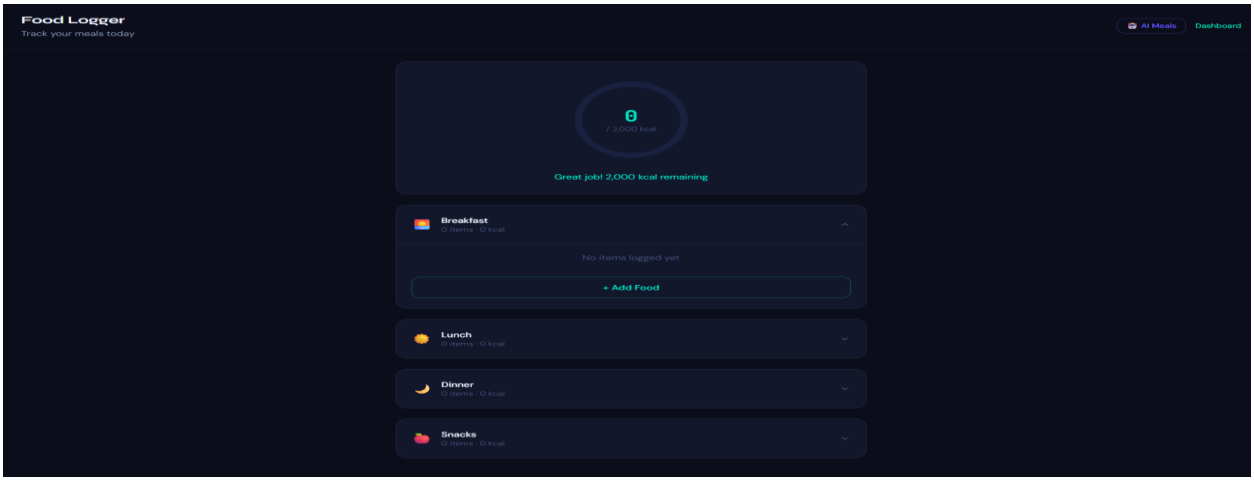


Figure 11 : Water Tracker



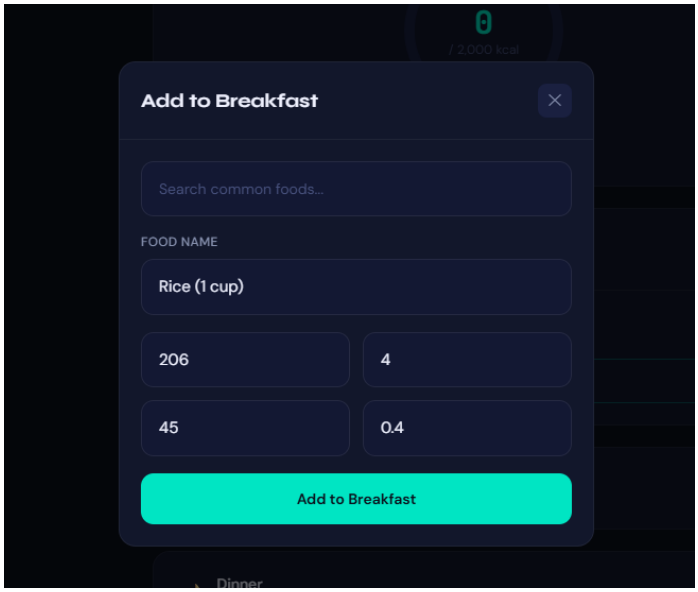


Figure 12 : Nutrition/Food Logger

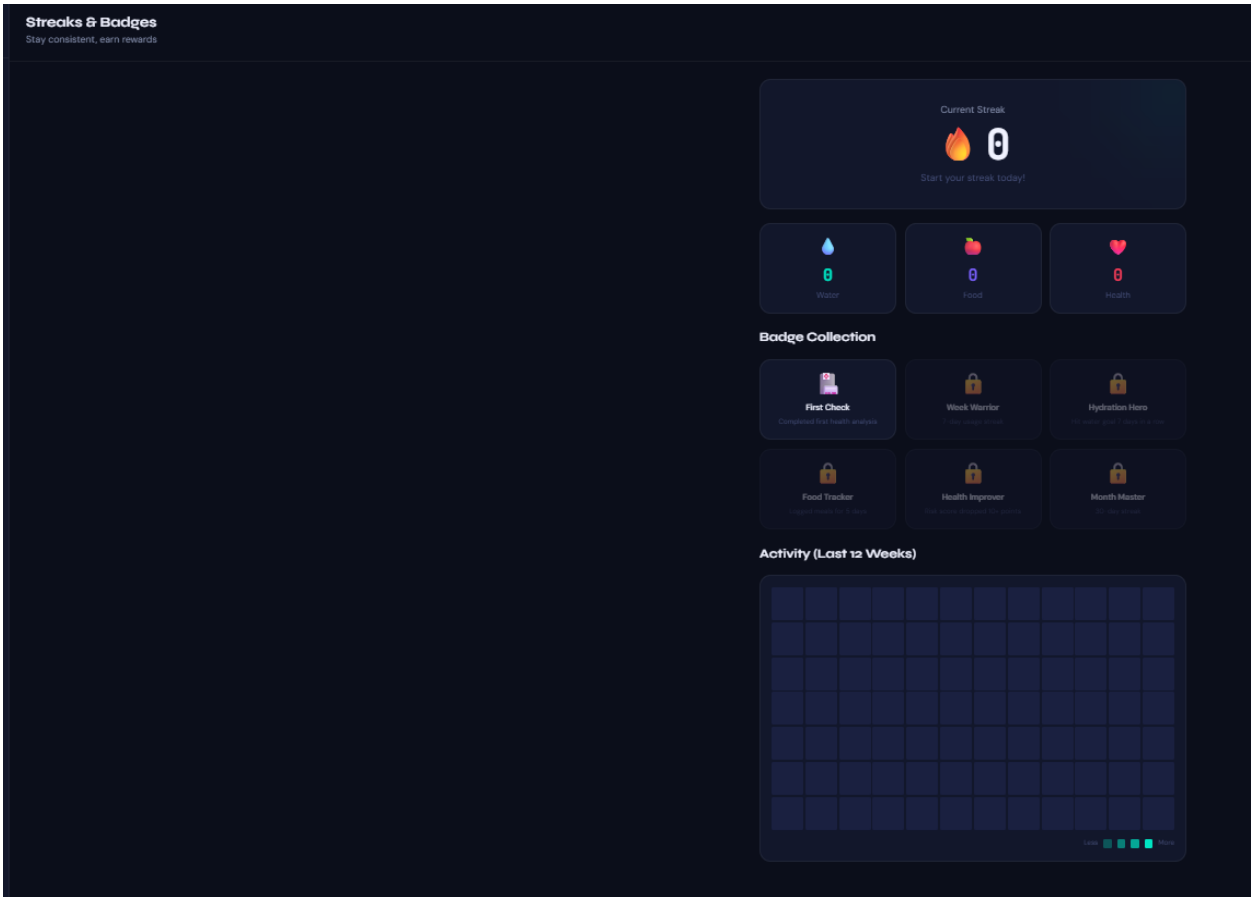


Figure 13 : Streak/Badges

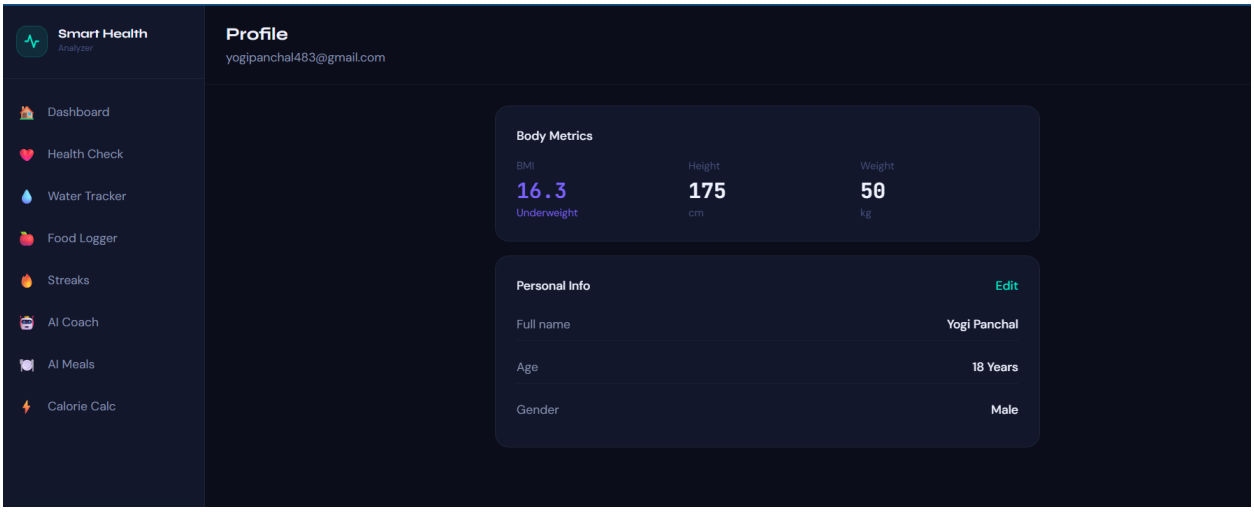


Figure 14 : Profile Page

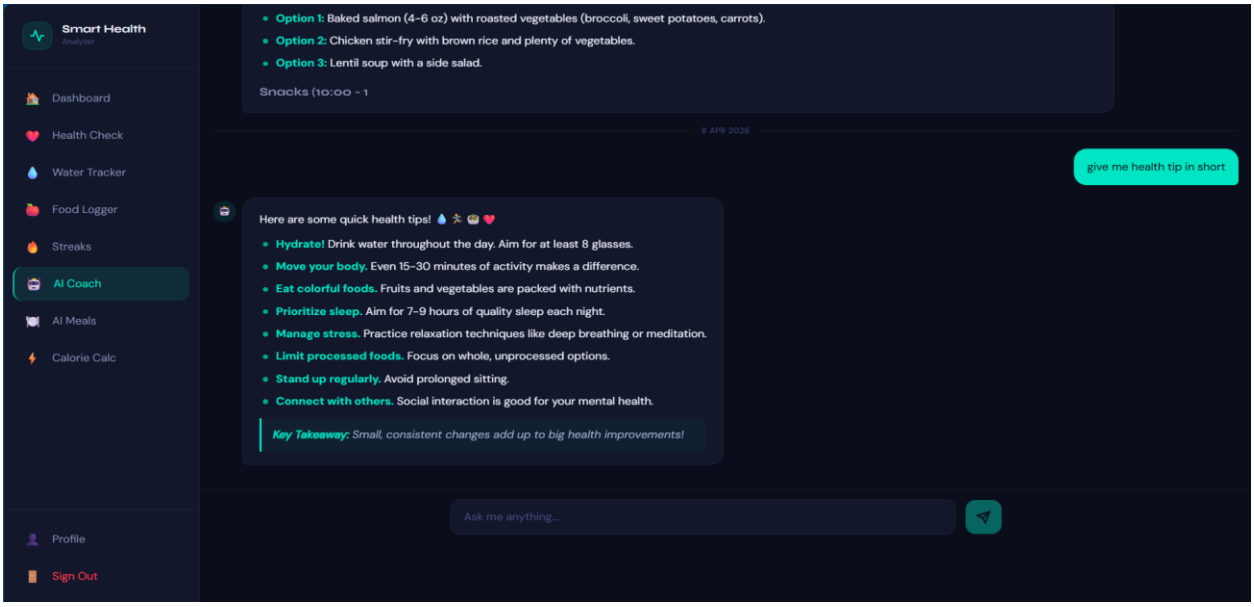


Figure 15 : Ai chat

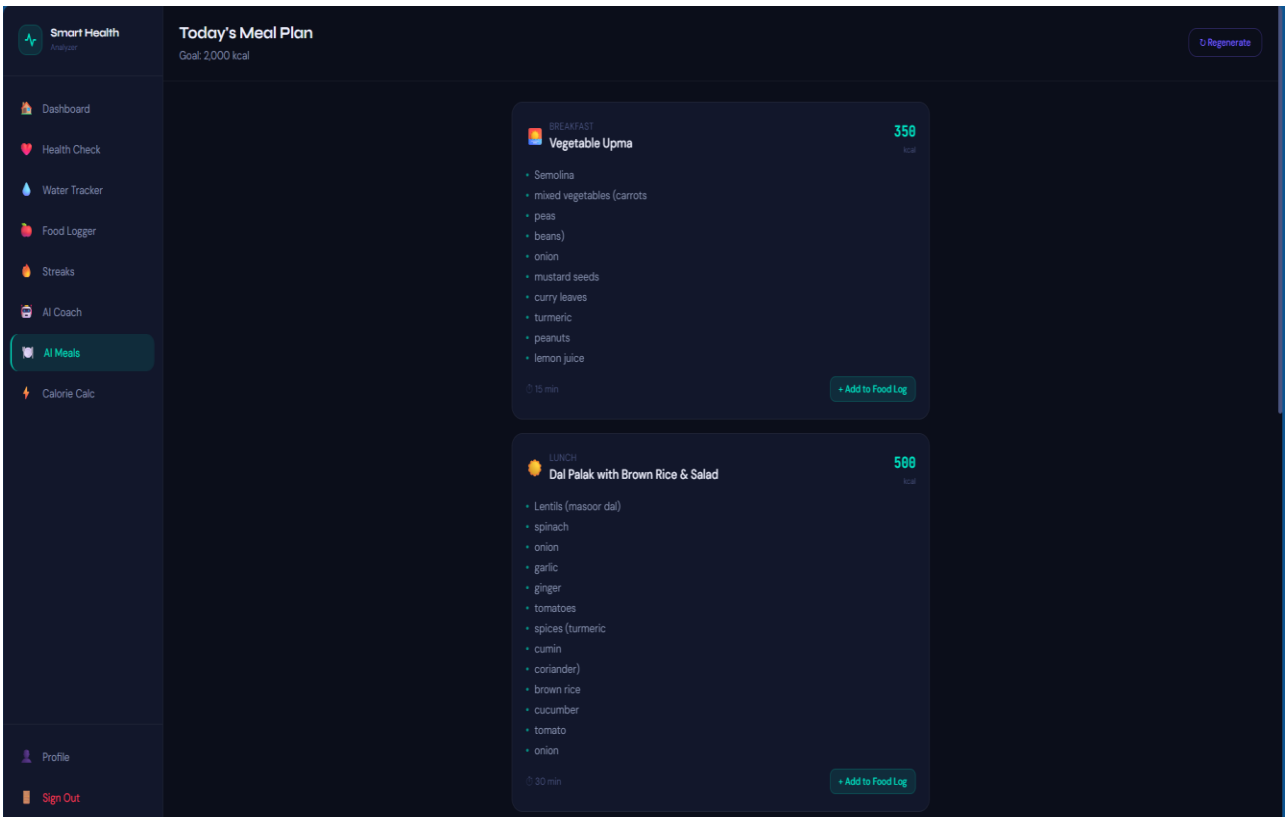


Figure 16 : Ai meal suggestion

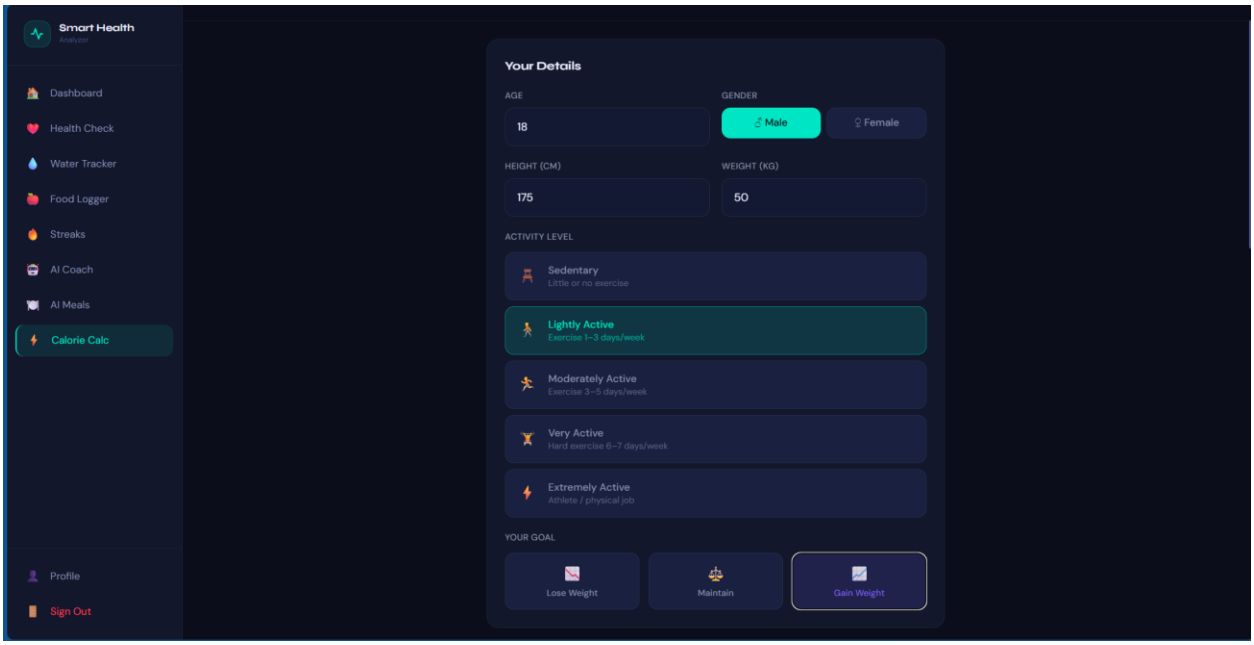


Figure 17 : Calorie Calculator

CHAPTER 8: CHALLENGES FACED

During the development of **Smart Health Analyzer**, several technical and architectural challenges were encountered:

Authentication Loading State Race Condition The most recurring challenge was ensuring that the Supabase auth session resolved fully before any navigation logic ran. If protected pages checked for an authenticated user before the auth state loaded, they would incorrectly redirect to the login page. This required careful implementation of an auth loading guard across all protected routes.

Supabase RLS Policy Syntax Configuring Row-Level Security policies in Supabase presented a challenge because the CREATE POLICY IF NOT EXISTS syntax is not supported in older PostgreSQL versions. Every schema migration required using a DROP POLICY IF EXISTS followed by CREATE POLICY pattern to avoid errors.

ML Model Serving on Free-Tier Infrastructure Deploying the scikit-learn models on Render's free tier introduced cold start delays of up to 30–50 seconds after inactivity. This required implementing skeleton loading UI on the frontend to prevent users from seeing blank screens or assuming the app had crashed.

Keeping RLS Consistent Across 10 Tables The profiles table uses id as its primary key for RLS comparisons, while all other tables use user_id. This inconsistency required careful attention in every new table's RLS policy to avoid silent data access failures.

Preventing Breaking Changes During Feature Additions As the project grew to include 10+ features, there was constant risk of new code breaking existing working functionality. This was managed by strictly following the principle of adding new files and routes rather than modifying existing working components.

CORS Configuration for Deployed Environments During deployment, the FastAPI backend's CORS configuration needed to be updated from localhost:5173 to the production Vercel URL. Forgetting this caused all prediction requests to fail in production while working locally.

CHAPTER 9: CONCLUSION AND FUTURE SCOPE

9.1 Conclusion

The **Smart Health Analyzer** project is a full-stack health monitoring and risk prediction platform that makes preventive healthcare accessible to everyday users. By combining **React 19, FastAPI, scikit-learn ML models, and Supabase**, it delivers clinical health risk screening for mental health, diabetes, and heart disease alongside daily wellness tracking features including water intake, nutrition logging, and a gamification-based streak system. The system is fully deployed, secure with row-level data isolation, and designed to be intuitive for non-technical users.

9.2 Future Scope

Future improvements planned for **Smart Health Analyzer** include:

- **Push Notifications:** Adding browser or mobile push notifications to remind users to log water, meals, and maintain their daily streak.
- **Mobile Application:** Developing a dedicated React Native mobile app to provide a native experience on iOS and Android.
- **Wearable Integration:** Connecting with health APIs (e.g., Google Fit, Apple Health) to automatically import step count, heart rate, and sleep data for richer health insights.

REFERENCES

1. **FastAPI Documentation** – https://fastapi.tiangolo.com/
2. **React Documentation** – https://react.dev/
3. **Supabase Documentation** – https://supabase.com/docs
4. **scikit-learn Documentation** – https://scikit-learn.org/stable/documentation.html
5. **Tailwind CSS Documentation** – https://tailwindcss.com/docs
6. **Vite Documentation** – https://vitejs.dev/guide/
7. **React Router v7 Documentation** – https://reactrouter.com/
8. **Axios Documentation** – https://axios-http.com/docs/intro
9. **Render Deployment Documentation** – https://render.com/docs
10. **Vercel Deployment Documentation** – https://vercel.com/docs
11. **UCI Machine Learning Repository (Heart Disease Dataset)** – https://archive.ics.uci.edu/ml/datasets/heart+disease
12. **PIMA Indians Diabetes Dataset** – https://www.kaggle.com/datasets/uciml/pima-indians-diabetes-database